

PlayerExpUpgradeWidget リファクタリング報告書

対象：経験値レベルアップ時の強化選択機能 / プランチ Todo/feature / 2026-07-16

1. 目的

強化選択UI（`PlayerExpUpgradeWidget`）に集中していたゲームロジックを分離し、「効果の種類が増えるほど適用処理が肥大化する」構造を、データ駆動の設計へ作り替えました。挙動（攻撃倍率の適用結果）は従来どおりに保ちつつ、拡張しても分岐が増えない形にしています。

2. 変更前の問題点

2-1. UI層にドメインロジックが書かれていた

Widget が本来「表示」だけを担うべきところで、以下まで抱えていました。

- DataTable（`DT_ExpUpgrade`）のロードとキャッシュ
- 強化候補のランダム抽選・重複排除
- 現在レベルの算出（`FCString::Atoi`）と行データの妥当性判定
- 攻撃倍率を取り出してランタイムデータへ受け渡し

2-2. 効果ごとのハードコード分岐 = 肥大化の温床

ユーザー指摘箇所（`// ここで攻撃力の乗数を...渡す`）の実体は、ランタイムデータ側の `UpgradeAttackMultiplier()` にある `if (Id == "0")` でした。

```
// 変更前：PlayerRuntimeData.cpp
void UPlayerRuntimeData::UpgradeAttackMultiplier(FName Id, float InMultiplier)
{
    for (auto& UpgradeState : UpgradeStates)
    {
        if (UpgradeState.UpgradeId == Id)
        {
            if (Id == "0")           // ← 効果ごとに分岐を足していく設計
            {
                float Multiplier = InMultiplier;
                UE_LOG(...);
                SetPlayerAttackDamage(Multiplier);
                break;
            }
        }
    }
}
```

攻撃力に加えて「速度」「HP」などを足すたびに `else if (Id == "1")` ... と分岐が増え、同時に Widget 側のマウス処理も選択肢の数だけコピペで膨らんでいく構造でした。

2-3. マウス処理が3枚分コピペで重複

```
// 変更前：PlayerExpUpgradeWidget.cpp (3ブロックほぼ同一)
if (UpgradeWidget_0->IsMouseOver()) {
    PlayerRuntimeData->UpdateUpgradeStates(UpgradeWidget_0->GetUpgradeRowName());
    PlayerRuntimeData->UpgradeAttackMultiplier(UpgradeWidget_0->GetUpgradeRowName(),
                                                UpgradeWidget_0->GetValidUpgradesMultiplier());
} else if (UpgradeWidget_1->IsMouseOver()) { /* 同じ処理 */ }
else if (UpgradeWidget_2->IsMouseOver()) { /* 同じ処理 */ }
```

3. 設計方針

方針	内容
責務分離	UI (Widget) は「渡された候補を表示」「選ばれた行名を通知」するだけ。抽選・レベル判定・効果適用はすべて <code>UPlayerRuntimeData</code> に集約。
データ駆動	「どのステータスに効くか」を DataTable の <code>EUpgradeEffectType</code> 列で表現。適用側は種別で振り分けるだけで、Id ごとの分岐を持たない。
効果値を種別マップで保持	ランタイムデータは <code>TMap<EUpgradeEffectType, float> EffectMultipliers</code> を1つ持つ。効果が増えなくても「保持する場所」は増えない。

4. 「肥大化」がどう解消されたか

変更前：効果を1つ足すと...

- `UpgradeAttackMultiplier` に `else if` 分岐を追加
 - 専用のセッター／メンバ変数を追加
 - Widget が倍率を保持・受け渡すコードを追加
 - 3箇所のマウス処理を確認・修正
- UI とロジック両方に手が入り、分岐と重複が増殖。

変更後：効果を1つ足すと...

- `EUpgradeEffectType` に列挙子を1つ追加
 - そのステータスの計算箇所では `GetEffectMultiplier(種別)` を参照
- 適用の振り分け (`ApplyUpgradeEffect`) ・Widget ・マウス処理は一切変更不要。

5. 主な変更コード

5-1. DataTable に効果種別を追加 (ExpUpgradeDataTable.h)

```
UENUM(BlueprintType)
enum class EUpgradeEffectType : uint8
{
    AttackDamage    UMETA(DisplayName = "攻撃力"),
    // 効果を増やすときはここに列挙子を1つ足すだけ
};

USTRUCT(BlueprintType)
struct FExpUpgradeLevelData
```

```

{
    GENERATED_BODY()
    UPROPERTY(EditAnywhere, BlueprintReadWrite) FText Description;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) EUpgradeEffectType EffectType = EUpgradeEffectType::AttackD
    UPROPERTY(EditAnywhere, BlueprintReadWrite) float AttackMultiplier = 1.0f; // EffectType への効果量
};

```

5-2. 適用処理をランタイムデータに集約 (PlayerRuntimeData.cpp)

```

// 選択された強化を適用 (レベル加算 + 効果反映)。Id ごとの分岐なし
void UPlayerRuntimeData::ApplySelectedUpgrade(FName Id)
{
    if (const FExpUpgradeLevelData* LevelData = FindCurrentLevelData(Id))
    {
        ApplyUpgradeEffect(LevelData->EffectType, LevelData->AttackMultiplier);
    }
    UpdateUpgradeStates(Id);
}

// 効果種別で振り分けるだけの共通処理
void UPlayerRuntimeData::ApplyUpgradeEffect(EUpgradeEffectType Type, float Value)
{
    EffectMultipliers.FindOrAdd(Type) = Value;
}

float UPlayerRuntimeData::GetEffectMultiplier(EUpgradeEffectType Type) const
{
    const float* Found = EffectMultipliers.Find(Type);
    return Found ? *Found : 1.0f;
}

// 攻撃力計算は種別マップを参照するだけ
Attack.Final = Attack.Base * GetEffectMultiplier(EUpgradeEffectType::AttackDamage);

```

5-3. UIは表示と通知に専念 (PlayerExpUpgradeWidget.cpp)

```

// 抽選はランタイムデータへ委譲。UIは受け取って並べるだけ
const TArray<FValidUpgradeInfo> Choices = PlayerRuntimeData->BuildUpgradeChoices(UpgradeChoiceCount);

// マウス処理：3枚のコピペ → ループ1本 + 単一呼び出し
UExpUpgradeWidgetBase* Widgets[] = { UpgradeWidget_0, UpgradeWidget_1, UpgradeWidget_2 };
for (UExpUpgradeWidgetBase* Widget : Widgets)
{
    if (Widget && Widget->IsMouseOver())
    {
        if (PlayerRuntimeData) PlayerRuntimeData->ApplySelectedUpgrade(Widget->GetUpgradeRowName());
        break;
    }
}

```

6. 変更ファイル一覧

ファイル	内容
ExpUpgradeDataTable.h	EUpgradeEffectType と EffectType 列を追加。UI用DTO FValidUpgradeInfo をドメイン側へ移動。
PlayerRuntimeData.h / .cpp	BuildUpgradeChoices / ApplySelectedUpgrade / ApplyUpgradeEffect / GetEffectMultiplier / FindCurrentLevelData を追加。if(Id=="0") の UpgradeAttackMultiplier と単一 AttackMultiplier を撤廃し、種別マップに置換。
PlayerExpUpgradeWidget.h / .cpp	抽選・DataTableロード・倍率受け渡しを削除。表示と選択通知のみに。未使用スタブ（SetAttackMultiplier 等）も除去。
ExpUpgradeWidgetBase.h	UIが倍率を保持する ValidUpgradesMultiplier と関連アクセサを削除。

7. 反映時の注意点

- ① **再ビルドが必要**：ヘッダ（USTRUCT/UENUM）を変更したため、エディタを閉じて C++ をリビルドしてください。
- ② **DataTable の設定**：DT_ExpUpgrade の各レベルに EffectType 列が増えます。既存行は既定値 AttackDamage になるため、攻撃力強化はそのまま動作します。別ステータスの強化を作る場合はこの列を設定してください。
- ③ **挙動の変化点**：旧コードは if(Id=="0") により行"0"だけ攻撃倍率が反映されていました。新実装は**全ての行**が自身の EffectType に沿って効果を反映します（＝本来あるべき挙動）。もし効果を持たせたくない行がある場合は、DataTable側で倍率を 1.0 にしてください。

以上。設計の要点は「UI = 表示」「ランタイムデータ = ロジック」「DataTable = 効果の定義」の3層分離と、効果種別による分岐レス化です。